192425192

G Vishnu Madhav

EXP18:

```
#include <stdio.h>

#include <string.h>


static const int
PC1[56]={57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,59,51,43,35,27,19,11,3,60,52,44,36,
63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,61,53,45,37,29,21,13,5,28,20,12,4};

static const int
PC2[48]={14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,26,8,16,7,27,20,13,2,41,52,31,37,47,5
5,30,40,51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32};

static const int SHIFTS[16]={1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};


int main() {
   int c[28], d[28], t[28];           /* hold the ORIGINAL key-bit numbers (1..64) */
   int sub[16][48];
   int inA[65] = {0}, inB[65] = {0};   /* key bits used in first / second 24 bits   */
   int r, i, s, cntA = 0, cntB = 0, common = 0, ok = 1;


   for (i = 0; i < 28; i++) { c[i] = PC1[i]; d[i] = PC1[28 + i]; }
   for (r = 0; r < 16; r++) {
      for (s = 0; s < SHIFTS[r]; s++) {
         memcpy(t, c, sizeof c); for (i = 0; i < 28; i++) c[i] = t[(i + 1) % 28];
         memcpy(t, d, sizeof d); for (i = 0; i < 28; i++) d[i] = t[(i + 1) % 28];
      }
      for (i = 0; i < 48; i++) {
         int pos = PC2[i] - 1;
         sub[r][i] = (pos < 28) ? c[pos] : d[pos - 28];
      }
      /* first 24 bits of the subkey must come from C-half, last 24 from D-half */
```

```
    for (i = 0; i < 24; i++) { inA[sub[r][i]] = 1; if (PC2[i] > 28) ok = 0; }
    for (i = 24; i < 48; i++) { inB[sub[r][i]] = 1; if (PC2[i] <= 28) ok = 0; }
  }

  printf("Key bits that feed the FIRST 24 bits of the 16 subkeys:\n  ");
  for (i = 1; i <= 64; i++) if (inA[i]) { printf("%d ", i); cntA++; }
  printf("\n  (%d bits, all taken from the 28 bits of C0)\n\n", cntA);
  printf("Key bits that feed the LAST 24 bits of the 16 subkeys:\n  ");
  for (i = 1; i <= 64; i++) if (inB[i]) { printf("%d ", i); cntB++; }
  printf("\n  (%d bits, all taken from the 28 bits of D0)\n\n", cntB);

  for (i = 1; i <= 64; i++) if (inA[i] && inB[i]) common++;
  printf("Bits common to both subsets: %d\n", common);
  printf("First 24 bits -> same 28-bit subset (C), second 24 bits -> disjoint\n");
  printf("28-bit subset (D): %s\n", (ok && common == 0) ? "VERIFIED" : "NOT TRUE");
  return 0;
}
```

OUTPUT:

```
Key bits that feed the FIRST 24 bits of the 16 subkeys:
  1 2 3 9 10 11 17 18 19 25 26 27 33 34 35 36 41 42 43 44 49 50 51 52 57 58 59 60
  (28 bits, all taken from the 28 bits of C0)

Key bits that feed the LAST 24 bits of the 16 subkeys:
  4 5 6 7 12 13 14 15 20 21 22 23 28 29 30 31 37 38 39 45 46 47 53 54 55 61 62 63
  (28 bits, all taken from the 28 bits of D0)

Bits common to both subsets: 0
First 24 bits -> same 28-bit subset (C), second 24 bits -> disjoint
28-bit subset (D): VERIFIED
```